Temporal Checkpoints & Explanations

> **One-line definition:**
> Temporal is a durable execution platform that turns ordinary Python functions into fault-tolerant, stateful, long-running processes — without you writing retry loops, state stores, or recovery logic.

Support Legend

Label	Meaning
Supported	Built into Temporal — zero extra infrastructure needed
Partially Supported	Works via design pattern — requires intentional code structure
Not Supported	Needs an external system; Temporal orchestrates it but doesn't provide it

---

Table of Contents

---

1. Loops Inside Temporal

Status: Supported

> **In one line:** Write a normal Python loop — Temporal makes it crash-proof by remembering every completed step.

How Temporal Handles It

- **Replay from event history** — When a Worker restarts, Temporal replays the workflow from its recorded history, picking up exactly where it stopped.
  - *Simply put: your loop resumes from the last completed iteration, not from the beginning.*
- **Activity results are cached** — Every completed Activity inside the loop is stored as an event. On replay, the cached result is injected — the Activity code never runs again.
  - *This means: if your server crashes on iteration 500, Temporal won't re-run iterations 1–499.*
- **Loop control via Signals** — Stop flags, max counts, and pause conditions live in Workflow state (`self.*`). Signals flip these flags from outside the running workflow.
- **`continue_as_new` for infinite loops** — For very long-running loops, use `continue_as_new` to carry forward only essential state and start a fresh event history.
  - *Why: Temporal's event history has a ~50k event ceiling. Infinite loops need this safety valve.*

Key Notes

- **The cached result is the core mechanism** — Temporal reads stored results during replay; it never makes the same call twice. This is what makes the loop durable.
- **`continue_as_new` is required in production infinite loops** — Not a nice-to-have. Design for it from the start.
- **Signals are the correct way to stop a loop** — Avoid `while True:` without a signal-controlled stop flag.

---

2. Retries

Status: Supported

> **In one line:** Activities automatically retry on failure — declare the policy once, Temporal handles back-off, tracking, and exhaustion.

How Temporal Handles It

- **Failure is recorded, not lost** — When an Activity fails, Temporal writes `ActivityTaskFailed` to the event history, then schedules a new attempt after the back-off interval.
- **Attempt tracking** — Each retry increments `activity.info().attempt`. The Activity code can read this to adjust behaviour on later attempts.
- **Invisible to the Workflow** — The Workflow sees only the final success. All intermediate failures are hidden.
  - *Simply put: the Workflow waits patiently; it doesn't know or care how many retries happened.*
- **Exponential back-off by default** — Default: `backoff_coefficient=2.0`, capped at 100× initial interval, unlimited attempts unless you set `maximum_attempts`.
- **`non_retryable=True` stops retries immediately** — Raising `ApplicationError(..., non_retryable=True)` signals a business logic error. Temporal stops retrying and fails the Activity.

Key Notes

- **Always set `maximum_attempts` for external service calls** — Unbounded retries can hammer a downstream API during an outage.
- **Only the Activity retries — the Workflow stays paused** — No part of the Workflow re-runs during Activity retries. It simply waits for the result.

- `non_retryable=True` **is the most important flag to get right** — Omitting it on validation errors burns retries needlessly and makes debugging painful.

---

3. Audit Functionality / Monitoring

Status: Supported (Event History) + Not Supported (Metrics/Alerting — requires external tooling)

> **In one line:** Every workflow step is written to an immutable log — inspectable in the Web UI, via CLI, or programmatically at any time.

How Temporal Handles It

- **Event history is the built-in audit trail** — Every `ActivityTaskScheduled`, `ActivityTaskCompleted`, `TimerFired`, `SignalReceived`, and more is recorded with full timestamps and payloads.
- **Temporal Web UI (`:8233`)** — Visualises the full event history graphically; shows current state, inputs, outputs, and failures at a glance. Your first stop for any investigation.
- **CLI inspection** — `temporal workflow show --workflow-id <id>` dumps the complete event log. Useful for scripting or deep debugging outside the UI.
- **Replay-safe structured logging** — `workflow.logger` and `activity.logger` suppress duplicate log entries during replay automatically.
  - *Why this matters: plain `print()` or `logging` would fire on every replay, flooding your log system.*
- **Heartbeats as sub-activity checkpoints** — Long-running Activities call `activity.heartbeat(progress)` mid-execution. If the Worker crashes, the next attempt can resume from the last checkpoint rather than restarting from zero.
- **External metrics via SDK** — Temporal exports Prometheus-compatible metrics (task queue depth, workflow latency, error rates). Connect to Datadog, Grafana, or PagerDuty separately.

Key Notes

- **Always use `workflow.logger` inside Workflows — never `print()`** — Replay-safety is not negotiable.
- **Heartbeats are your sub-activity audit trail** — For any Activity running more than a few seconds, heartbeat regularly to capture progress.
- **The Web UI is your first debugging tool** — Before writing diagnostic code, check the event history in the UI. The answer is almost always already there.

---

4. Trigger Mechanisms

Status: Supported (Signals, Schedules, Child Workflows) + Not Supported (raw HTTP/event triggers — your code bridges those)

> **In one line:** Workflows can be started by code, on a schedule, by another workflow, or by any external event your application code handles.

How Temporal Handles It

Trigger Type	How
From application code	`client.start_workflow()` or `client.execute_workflow()`
On a schedule	`CronSchedule` parameter or the Temporal Schedules API
Signal (into running workflow)	`handle.signal(workflow.some_signal, payload)`
From a parent workflow	`workflow.start_child_workflow()`
From an external event (Kafka, webhook)	Your consumer calls `client.start_workflow()`

- **Temporal Schedules** (the modern API) support pause/unpause, backfill for missed runs, jitter, and timezone awareness — far more powerful than a plain cron.
- **Signals vs. triggers** — Signals message an already-running workflow; they do not start new ones.
  - *To start from a Kafka event: your consumer receives the message and calls `client.start_workflow()`.*

Key Notes

- **Temporal has no native HTTP endpoint for triggers** — That is your application layer. Temporal handles everything after the start call.
- **Prefer the Schedules API over `cron_schedule`** — It handles missed runs, overlapping executions, and pausing; plain cron does not.
- **Use Workflow IDs as idempotency keys** — Pass a deterministic `id` to `start_workflow()` so duplicate trigger events don't spawn duplicate workflows.

---

5. Data Capture

Status: Supported (Event History Payloads) + Partially Supported (Long-term / domain data — requires Activity writing to external store)

> **In one line:** Every input, output, and signal payload is automatically serialised into the event history — readable at any time without extra setup.

How Temporal Handles It

- **DataConverter serialises everything** — Workflow inputs, Activity inputs/outputs, signal payloads, and query results are all serialised (JSON by default) into the event history automatically.
- **Typed dataclasses work natively** — Use `@dataclass` for all I/O types. Temporal serialises and deserialises them automatically, giving structured, type-safe payloads.
- **Search Attributes for cross-workflow queries** — Tag executions with custom indexed metadata (e.g., `customer_id`, `order_status`) and query across all running or completed workflows using those attributes.
- **Long-term storage goes through an Activity** — For data warehouses, analytics DBs, or audit tables that outlast the workflow, write to them via an Activity.

Key Notes

- **The event history is not a database** — It stores execution data, not business/domain data. Do not treat it as a query-able store for reporting or analytics.
- **Use typed dataclasses, not raw dicts** — `dict` payloads lose type safety and make the Web UI payload view harder to read and understand.
- **Encrypt sensitive payloads with a custom DataConverter** — PII must not appear in plain text in the event history or Web UI. Custom DataConverters add encryption at rest.

---

6. Validation and Correctness

Status: Partially Supported

> **In one line:** Temporal doesn't validate inputs for you — but it gives you precise tools to fail fast on bad data without wasting retries.

How Temporal Handles It

- **Activity-level validation with `ApplicationError`** — Raise `ApplicationError("message", non_retryable=True)` for business logic errors. Temporal stops retrying immediately.
  - *Distinction: transient errors (network, timeout) should retry; invalid input should not.*
- **Typed dataclasses enforce structure** — Python's type system catches shape mismatches at serialisation time when you use typed annotations on all dataclass fields.
- **Update validators (`@workflow.update_validator`)** — Declare a validator for an Update handler. If it raises, the Update is rejected before it touches any Workflow state.
  - *The Workflow never sees the invalid update — it's rejected at the boundary.*
- **Workflow-level guards** — Check preconditions near the top of `@workflow.run` before dispatching Activities. Fail fast if required state isn't present.

Key Notes

- **Temporal cannot validate data before a Workflow starts** — Pre-start validation is the caller's responsibility, or an initial validation Activity.
- **`non_retryable=True` is the single most important correctness tool** — It separates "retry this" from "stop and surface the error."
- **Update validators are underused** — They are the cleanest way to enforce invariants on external inputs to a running workflow without writing try/except inside the handler.

---

7. Decision and Routing

Status: Supported

> **In one line:** Plain Python `if/elif/else` inside a Workflow is your router — and Temporal makes those decisions durable.

How Temporal Handles It

- **Workflow code is the routing logic** — `if/elif/else` determines which Activities execute, in what order, and with what parameters. No separate routing engine needed.

- **Route on Activity results** — Query a DB, call an API, get the response via an Activity, then branch based on the result. The routing decision is recorded in event history.
  - *Simply put: never route on data you read directly in the Workflow. Route on what an Activity returned.*
- **Signals inject routing decisions at runtime** — An operator or external system can send a Signal to change the direction of a live, running workflow.
- **Child Workflows route entire sub-processes** — Spawn a child on a different Task Queue or Worker pool for isolation, separate retry scope, or dedicated resources.
- **All routing is automatically durable** — The routing decision is part of the event history; on replay, the same branch is always taken.

Key Notes

- **All routing logic must be deterministic** — Never branch on `random.random()`, `datetime.now()`, or direct network calls inside the Workflow. Drive all non-deterministic decisions through Activity results.
- **Signals are a powerful runtime routing tool** — They let external systems change course in a workflow that is already mid-execution.
- **Child Workflows give routing with full isolation** — If a sub-process needs its own retry policy, timeout, or resource pool, a dedicated child workflow is cleaner than cramming extra Activities into the parent.

---

8. Human Interaction

Status: Partially Supported

> **In one line:** Temporal can pause a workflow for days waiting for a human — the wait is crash-proof and costs zero compute.

How Temporal Handles It

- `workflow.wait_condition` — The Workflow suspends at a condition. No thread is blocked, no resource is consumed during the wait.
  - *Even if every server in your cluster restarts during the wait, the workflow resumes exactly where it paused.*
- **Signal wakes the workflow** — When a human acts (clicks "Approve," submits a form), your backend calls `handle.signal(workflow.approve, reviewer_id)`. The condition resolves and execution continues.
- **Queries provide live status to the UI** — `handle.query(workflow.is_pending)` returns current state without modifying anything. No polling loop, no DB read.
- **Configurable wait timeout** — Pass a `timeout` to `wait_condition` so workflows auto-expire if no human responds within N days.
- **You provide the UI** — Temporal has no built-in approval UI. Your frontend, form, or admin panel is responsible for delivering the Signal.

Key Notes

- **The durable wait is the key differentiator from polling** — A cron job checking a DB every minute burns resources; `wait_condition` costs nothing until it fires.
- **Always set a timeout on human-interaction waits** — Leaving it infinite means a forgotten approval permanently blocks the workflow and its resources.
- **Queries are always safe for UI polling** — Read-only, synchronous, and never affect workflow state.

---

9. Execution (External Systems)

Status: Supported (via Activities) + Not Supported (the external systems themselves)

> **In one line:** Activities are the bridge to the outside world — every API call, DB write, or email goes through an Activity, never directly from the Workflow.

How Temporal Handles It

- **Workflow → Activity → external system** — The Workflow never touches the network directly. It calls `workflow.execute_activity(...)` and waits for the result.
- **Activity runs on a Worker** — The Worker has full access to the network, DB drivers, HTTP clients, file system, etc. All real I/O happens here.
- **Success is recorded permanently** — Temporal writes the Activity result to the event history on completion. On replay, this result is returned directly — the external system is never called again.
- **Failures retry automatically** — If the external call fails (timeout, 5xx error), Temporal retries per the `RetryPolicy`. The Workflow only sees the eventual success.
- **Class-based Activities for shared connections** — Inject a shared HTTP session or DB connection pool into an Activity class. The Worker reuses the connection across all Activity invocations.

Key Notes

- **Activities must be idempotent** — Temporal may retry after a partial success (e.g., the Worker crashed after the API responded but before the result was recorded). Design external calls to be safe when called twice — use idempotency keys.
- **Set `heartbeat_timeout` on long-running Activities** — This tells Temporal how quickly to detect a stuck Worker and retry on another. Without it, a dead Worker can silently stall a workflow indefinitely.
- **Never call an external API directly inside the Workflow function** — It breaks determinism and destroys the replay guarantee.

---

10. Sequential vs Parallel Execution

Status: Supported

> **In one line:** `await` Activities one-by-one for sequential work; `asyncio.gather()` them for parallel — both are fully durable.

How Temporal Handles It

- **Sequential** — `await workflow.execute_activity(A)` then `await workflow.execute_activity(B)`. B does not start until A completes.

- **Parallel via `asyncio.gather`** — Schedule multiple Activities simultaneously. The Workflow resumes only when all of them complete.
  - *Use this for independent tasks that don't need each other's results to start.*
- **Parallel via `start_activity`** — Returns a handle immediately without blocking. Launch many Activities, collect handles, and await them all later. Ideal for dynamic fan-out driven by signals.
- **Child Workflows in parallel** — Start multiple child workflows and `asyncio.gather` their handles. Best when sub-processes need independent retry scopes or separate timeouts.
- **All patterns are fully durable** — Temporal tracks every `ActivityTaskScheduled` and `ActivityTaskCompleted` event, regardless of whether the pattern is sequential or parallel.

Key Notes

- **Do not use `asyncio.create_task()` or threads inside Workflows** — Both bypass Temporal's event loop control and break determinism, causing replay errors.
- **`asyncio.gather` inside a Workflow is deterministic** — Temporal's sandboxed event loop ensures the same scheduling order on every replay.
- **Sync Activities run on a thread pool** — They execute with true CPU parallelism, but from the Workflow's perspective they're identical to async Activities.

---

11. Stateful Workflow

Status: Supported

> **In one line:** Workflow state lives in `self.*` instance variables — Temporal automatically reconstructs them after any crash.

How Temporal Handles It

- **State = Python instance variables** — Whatever you store in `self._count`, `self._phase`, `self._items`, etc., is your workflow's persistent state.
- **Automatic reconstruction** — On Worker restart, Temporal replays the full event history. `__init__` runs first, then every recorded event is applied in order, restoring all `self.*` fields to their exact pre-crash values.
  - *Simply put: no database, no Redis, no manual serialisation — Temporal rebuilds state from history.*
- **Signals mutate state from outside** — `@workflow.signal` methods are the primary way to change state on a running workflow from external code.
- **Queries read state safely** — `@workflow.query` methods return snapshots without modifying anything. Safe to call at any time from any external code.

Key Notes

- **Never write to a DB directly inside `@workflow.run`** — Workflow code replays on every Worker restart. A DB write inside the Workflow body executes again on every replay, causing double-writes. All side effects must go through Activities.
- **The event history is your state store** — You don't need a separate persistence layer to remember where the workflow is.
- **Keep state minimal** — Large `self.*` objects (e.g., storing thousands of records) inflate the event history payload size. Store IDs and fetch data in Activities instead.

12. Deterministic Execution (Idempotency)

Status: Supported (Enforced by Design)

> **In one line:** Temporal replays your Workflow code to recover state — so the code must produce the exact same decisions every single run.

How Temporal Handles It

- **Sandboxed event loop** — Temporal runs Workflow code in a controlled environment that intercepts and blocks non-deterministic operations before they hazard replay.
- **Activity results injected, not re-run** — During replay, when the code reaches `execute_activity`, Temporal injects the previously recorded result. The Activity code never executes again.
  - *Simply put: the Activity code is skipped; only its stored result is replayed.*
- **`asyncio.sleep()` becomes a durable timer** — It is recorded as `TimerStarted`/`TimerFired` in the event history. During replay it is skipped instantly — no real waiting.
- **`workflow.now()` for safe time reads** — Returns the timestamp from the event history, not the system clock. Always use this inside Workflows instead of `datetime.now()`.
- **Non-determinism causes `WorkflowTaskFailed`** — If a code change alters the event sequence for an already-running workflow, Temporal raises a non-determinism error during replay.

Key Notes

- **Determinism is required for correctness, not optional** — A non-deterministic Workflow will fail during replay with a history mismatch error.
- **Three things never to do inside a Workflow:** call `random.random()`, call `datetime.now()`, make direct network calls. Route all of these through Activities.
- **Use `workflow.patched()` when changing live workflows** — It lets old and new execution paths coexist in the same codebase during a rolling deployment.

---

13. Exception Handling

Status: Supported

> **In one line:** Temporal has a clear error taxonomy — transient failures retry, business errors fail fast, and cancellations clean up gracefully.

How Temporal Handles It

Error Type	How to Signal It	Temporal Behaviour
Transient failure (network, timeout)	Any `Exception`	Retried per `RetryPolicy`
Business logic error	`ApplicationError(non_retryable=True)`	Fails immediately, no retry
Retries exhausted	Activity hits `maximum_attempts`	Workflow receives `ActivityError` and

Error Type	How to Signal It	Temporal Behaviour
		transitions to `Failed`
Graceful cancellation	`CancelledError` (from `cancel()`)	Workflow can run cleanup logic, then transitions to `Cancelled`
Hard stop	External `terminate()` call	Immediate stop — no cleanup window, no try/finally

- **`ActivityError` wraps the root cause** — When an Activity fails, the Workflow catches an `ActivityError`. Inspect `.cause` to access the original exception from the Activity code.
- **Raising inside Workflow code fails the Workflow immediately** — Unlike Activities, exceptions raised directly in `@workflow.run` do not retry. The Workflow transitions to `Failed`.
  - *Use this intentionally when a workflow-level condition is genuinely unrecoverable.*
- **Cancellation is cooperative** — The Workflow receives a `CancelledError` at the next `await` point and can run compensating logic (undo steps, notifications) before finishing.

Key Notes

- **The `ActivityError` wrapper is intentional** — It carries metadata (workflow ID, activity type, attempt number) alongside the root cause. Don't swallow it without inspecting `.cause`.
- **Design for cancellation** — Wrap cleanup logic in a `try/finally` block inside the Workflow. `CancelledError` is delivered at the next `await` point.
- **Termination is a last resort** — Use `cancel()` for graceful shutdown. Only use `terminate()` when the workflow is completely unresponsive.

---

14. Versioning

Status: Supported (Patching API) + Partially Supported (Requires code discipline during rolling deployment)

> **In one line:** You can safely change running Workflows without breaking them — `workflow.patched()` lets old and new code paths coexist during deployment.

How Temporal Handles It

- **The core problem** — Changing Workflow code while executions are in-flight causes replay errors, because the new code produces a different event sequence than what history recorded.
- **`workflow.patched("patch-id")`** — Returns `True` for new executions (post-deploy) and `False` for old ones replaying history. Use this to branch between old and new behaviour in the same codebase simultaneously.
- **`workflow.deprecate_patch("patch-id")`** — Once all pre-patch executions have completed, switch to this call to mark the old path as deprecated and signal intent to remove it.
- **Safe deployment sequence:**
  1. Deploy code with `workflow.patched()` — both old and new paths active
  2. Wait for all pre-patch executions to complete
  3. Remove the old path, deploy with `workflow.deprecate_patch()`
  4. Wait again, then remove all patching code entirely

Key Notes

- **This is the most common production mistake in Temporal** — Deploying a sequence change without `patched()` causes `WorkflowTaskFailed` non-determinism errors on all in-flight executions.
- **Patching is for sequence changes only** — Adding, removing, or reordering Activities, timers, and signals requires patching. Changing the internal logic inside an Activity function does not.
- **Never rename an Activity function referenced in in-flight history** — That also counts as a non-determinism change and will break replaying executions.

---

15. Resilience, Traceability, Reconstruction, Controllability

Status: Supported (All Four)

> **In one line:** Temporal workflows automatically survive failures, record every step, reconstruct their state, and accept external control — all built in, zero extra infrastructure.

How Temporal Handles It

Property	Mechanism	What It Means in Practice
Resilience	Event history + automatic replay	Crashes don't lose progress — the workflow restarts exactly where it stopped
Traceability	Immutable event log with full payload visibility	Every action, every failure, every input is permanently recorded and inspectable
Reconstruction	Deterministic replay restores exact state	No manual recovery logic — Temporal rebuilds all workflow state from history alone
Controllability	Signals, Queries, Updates, `cancel()`, `terminate()`	External code can read, influence, or stop any workflow at any time

- **Resilience** — Worker crashes, network partitions, and cluster restarts resolve transparently. The next available Worker picks up the workflow by replaying history. No data is lost.
- **Traceability** — The Web UI shows a timeline of every event with timestamps and payloads. The CLI exports it for scripting. Nothing is ever lost or overwritten.
- **Reconstruction** — Deterministic replay means Temporal never needs a snapshot or checkpoint file. History alone is sufficient to rebuild exact state at any point in time.
- **Controllability** — Queries are read-only and synchronous. Updates are synchronous with a response. Signals are asynchronous state mutations. `cancel()` is graceful, `terminate()` is immediate.

Key Notes

- **Queries never block the workflow** — They return immediately and are safe to call from monitoring dashboards, status APIs, or any external code.
- **Cancellation is cooperative; termination is not** — Always prefer `cancel()` so the workflow can run cleanup. Use `terminate()` only when the workflow is completely unresponsive.
- **These four properties together are what separate Temporal from a job queue** — A job queue gives you scheduling. Temporal gives you resilience + traceability + reconstruction + control as a single,

integrated guarantee.

---

16. Batch Execution

Status: Partially Supported

> **In one line:** Temporal handles durable, retryable batch orchestration — but it is not a data-processing engine like Spark or Flink.

How Temporal Handles It

- **Fan-out with `asyncio.gather`** — A parent Workflow starts one Activity (or child Workflow) per batch item simultaneously, collects handles, and awaits all results.
- **Signal-fed dynamic batches** — A long-running Workflow accepts item IDs via signals. Each signal adds to an internal queue; the Workflow launches an Activity per item with `start_activity`.
- **`continue_as_new` for large batches** — Process items in chunks. When a chunk completes, carry forward remaining work and restart with a fresh event history to stay under the event ceiling.
- **Temporal Schedules for periodic batch jobs** — Replace cron with Temporal Schedules for nightly / hourly batch runs. Get pause/resume, backfill, and execution overlap control.

Key Notes

- **Temporal is not Spark or Flink** — It excels where each item may need retries, human approval, or external API calls. It does not do high-throughput numeric computation over millions of rows.
- **`continue_as_new` is required for very large batches** — The ~50k event history limit is a hard ceiling. Design batch workflows with chunking from the start.
- **Each batch item gets independent retry** — Unlike a traditional batch job where one failure may abort everything, Temporal retries each item independently within its own Activity scope.

---

17. Real-time Execution

Status: Partially Supported

> **In one line:** Temporal supports near-real-time (50–200ms latency) but is not designed for sub-millisecond or high-frequency scenarios.

How Temporal Handles It

- **Inherent dispatch latency** — Every Activity passes through: Client → Cluster (gRPC) → Task Queue → Worker → execute → result back. Typical well-tuned round-trip: 50–200ms.
- **Updates for synchronous response** — `workflow.execute_update(...)` gives the caller a result back once the Activity completes, without any polling loop. The tightest synchronous interface Temporal offers.
- **Local Activities for sub-queue speed** — `workflow.execute_local_activity(...)` runs the Activity directly on the Worker, bypassing the cluster round-trip entirely.
  - *Trade-off: local Activities lose independent retry scope and are not visible as separate events in history.*

- **Reduce latency via co-location** — Running Workers on the same machine or in the same datacenter as the Temporal cluster significantly reduces dispatch overhead.

Key Notes

- **Temporal optimises for reliability, not raw speed** — For sub-100ms SLAs at high throughput, put a cache or low-latency service in front and use Temporal only for the durable orchestration layer.
- **Local Activities are not a general-purpose speed-up** — Only use them for in-process operations: read-only cache checks, fast transforms, lightweight validations. Do not use them for calls to external services.
- **Updates are the best real-time interface Temporal offers** — They provide a request-response contract with the running workflow without any polling overhead.

Summary Table

#	Checkpoint	Status
1	Loops Inside Temporal	Supported
2	Retries	Supported
3	Audit Functionality / Monitoring	Supported + Not Supported (Metrics)
4	Trigger Mechanisms	Supported + Not Supported (HTTP/Events)
5	Data Capture	Supported + Partially Supported (Custom Storage)
6	Validation and Correctness	Partially Supported
7	Decision and Routing	Supported
8	Human Interaction	Partially Supported
9	Execution (External Systems)	Supported + Not Supported (The Systems)
10	Sequential vs Parallel Execution	Supported
11	Stateful Workflow	Supported
12	Deterministic Execution (Idempotency)	Supported
13	Exception Handling	Supported
14	Versioning	Supported + Partially Supported
15	Resilience, Traceability, Reconstruction, Controllability	Supported
16	Batch Execution	Partially Supported
17	Real-time Execution	Partially Supported

Where Temporal Excels

These are the scenarios where Temporal provides the highest value and replaces significant custom infrastructure:

Scenario	Why Temporal Wins
Long-running, multi-step business processes	Crash-proof execution with zero manual state management
Automatic retry without code	Declare policy once; Temporal handles back-off and exhaustion
State management without a database	`self.*` fields survive crashes and restarts automatically
Distributed saga / compensation patterns	`try/finally` + Activities = durable compensating transactions
Human-in-the-loop approval workflows	Crash-proof `wait_condition` replaces polling, queues, and DB flags
Complete audit trail	Immutable event history requires zero extra setup
Parallel + sequential orchestration	`asyncio.gather` and `start_activity` both fully durable
Workflow versioning with live traffic	`workflow.patched()` lets old and new paths coexist safely
Testing (time skipping, mocking)	Built-in test environment for fast, deterministic unit tests

Where NOT to Use Temporal

Understanding the boundary is equally important. Using Temporal outside its design intent adds complexity without benefit:

System	Why Temporal Doesn't Replace It
Message queues (Kafka, RabbitMQ)	High-throughput, fire-and-forget event streaming at millions/sec
Databases	Complex ad-hoc queries, joins, long-term business data storage
Stream processors (Flink, Spark)	Sub-millisecond stateless transforms over massive data volumes
API gateways	Request routing, authentication, rate limiting
Simple schedulers (cron)	Time-based triggers with no stateful orchestration needed
Monitoring systems (Datadog, Prometheus)	Metrics aggregation, anomaly detection, alerting dashboards
Real-time bidding / trading systems	Sub-millisecond latency requirements

One-Line Closing Definition

> **Temporal makes your code durable** — if a server crashes in the middle of your business process, Temporal picks up from exactly where it left off, without you writing a single line of retry, state recovery, or distributed transaction logic.